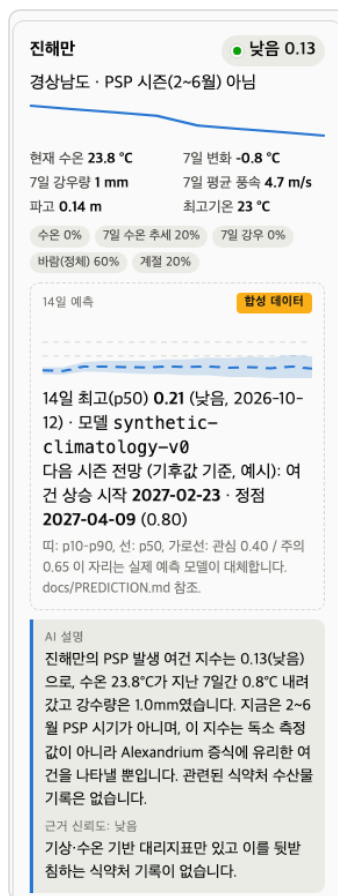


From Index to Model

A Roadmap for Predicting Shellfish Toxin Events

지수에서 모델로: 패류독소 발생 예측을 위한 로드맵

What the K-Food Safety Watch repository already gives you, what a real predictive model must produce, how to get from a weather heuristic to a backtested forecast, and which parts of that work belong to the scientist, the engineer and the AI.



Written in Seoul, 9 October 2026, after the afternoon session with the colleague who will lead this work Bob Currier (OCEANCODA) with Claude Code

Repository: github.com/robertdcurrier/korean_food_safety · this document is [docs/Shellfish-Prediction-Roadmap.pdf](#) ; the primary guide is [docs/K-Food-Safety-Watch-Guide.pdf](#)

Contents

1. Why this document exists
2. What exists today: three phases in one afternoon
3. Interface first: the forecast contract
4. Define the target before touching data
5. Data inventory: labels and features
6. The modelling ladder
7. Evaluation is the scientist's job
8. Communicating a forecast
9. Where Claude helps, and where it does not
10. Team, roles and a twelve-week plan
11. Checklist before switching off the SYNTHETIC badge
12. Glossary

SIXTY-SECOND VERSION

The repository now has a slot for a forecast: a fixed JSON shape per bay, a page that draws it with an uncertainty band, and tests that enforce it. A synthetic generator fills the slot today and is badged SYNTHETIC everywhere. Your team's job is to replace one function with a model that has been backtested against real NIFS toxin history. Before any modelling, write a one-page target specification: what counts as an event, at which stations, with what lead time. Climb the ladder from climatology to a heuristic to logistic regression to gradient boosting, and never let a rung ship unless it beats the one below on the same backtest. The scientist owns the target, the labels and the evaluation; the engineer owns the data pipes; Claude Code builds and documents; the Claude API explains. The statistical model is ordinary statistics, not a language model.

1. Why this document exists

The primary guide explains how the K-Food Safety Watch app was built and how to run it. It is for one person learning a working method. This document is different. It is for a team that will try to predict shellfish toxin events, and it is about the gap between the proof of concept on the page and a forecast someone could act on.

The request that produced it, verbatim:

"My colleague is interested in building a predictive model for shellfish events. While that is too much for this session, can we add output to the map data for each Bay with a synthetic prediction. We'll also need to include this as another training document, distinct from the primary one."

Read it alongside three files in the repository: `docs/ENVIRONMENT.md` (the conditions index), `docs/PREDICTION.md` (the forecast contract), and `src/kfs/predict.py` (the synthetic filler). Nothing here requires you to read code, but every claim here can be checked against those files.

THE LESSON CARRIED OVER FROM THE PRIMARY GUIDE

Own the question, not the keyboard. In a prediction project that means: the scientist writes the target specification, chooses the labels, and judges the backtest. Everything else can be delegated, including to Claude. Nothing in this document changes that; it just shows how much of the question there is to own.

2. What exists today: three phases in one afternoon

Phase	What it does	What it teaches
1 · Recalls	MFDS recalls and failed inspections, enriched by Claude, on a bilingual map and list.	Public API, one AI judgement step, deterministic code for the rest, a local HTML result, a repo that explains itself.
2 · Environment	Eight south-coast bays scored for bloom-favourable conditions from seven days of Open-Meteo sea temperature, rainfall and wind, with a Claude advisory per bay.	A heuristic index is a legitimate first model if it is explicit, tested and honest about what it is not. In October it says "low, out of season", which is the right answer.
3 · Forecast slot	Each bay carries a 14-day forecast of the index with a p10–p90 band and a next-season outlook, generated synthetically and badged as such.	Build the interface before the model. The page, the data file, the tests and the documentation now all exist. The model replaces one function.

The screenshot on the cover is one bay card. Reading it top to bottom: observed conditions and a sparkline of the week's sea temperature; the five factors of the index; the forecast block with its dashed line, shaded band and SYNTHETIC badge; the Claude advisory, which deliberately does not see the synthetic forecast.

WHAT PHASE 3 IS NOT

It is not a prediction. The synthetic generator blends today's observed sea temperature toward a climatology, lets rainfall relax toward its monthly mean, assumes wind persistence, and scores each future day with the same index as the tab. It will look plausible. It has no skill, and it has never been compared to a single real toxin event. That is why it is badged, why the model name is printed on every card, and why the advisory prompt is not allowed to read it.

3. Interface first: the forecast contract

Every bay's forecast is one JSON object with a fixed shape. The page draws it, the tests enforce it, and the model you build must produce it.

```
{
  "model": "synthetic-climatology-v0",
  "issued_at": "2026-10-09",
  "synthetic": true,
  "horizon": [
    {"date": "2026-10-10", "p10": 0.07, "p50": 0.14, "p90": 0.20, "level": "low"},
    ... fourteen daily points ...
  ],
  "season_outlook": {
    "first_favourable": "2027-02-23",
    "peak": "2027-04-09",
    "peak_index": 0.80,
    "basis": "climatology only; illustrative"
  }
}
```

Rules

- **Quantiles, not a point.** `p10`, `p50`, `p90` are quantiles of the conditions index, or later of the event probability, in $[0, 1]$. A model that cannot express uncertainty emits three equal values and says so in `model`. The page will draw a line with no band, which is itself information.
- `level` **derives from** `p50` with the thresholds the tab already uses: 0.40 moderate, 0.65 elevated. When the target changes from "conditions" to "event probability", the thresholds change and the expert sets them.
- `synthetic` **stays true** until a model with a documented backtest ships. The badge is driven by this flag alone.
- `model` **names the exact version** and is printed on the card. Anyone looking at the page can tell which model produced what they see.
- **The page knows nothing about how the object was made.** That is the point. Replace `kfs.predict.forecast` and nothing else moves.

WHY BUILD THE INTERFACE BEFORE THE MODEL

Because it lets five things proceed in parallel instead of in sequence: the page design, the documentation, the tests, the evaluation harness, and the data pipelines. It also forces a decision that modelling teams often postpone until it is painful: what, exactly, is the output? Fourteen daily quantiles per bay is a decision. Having made it in an afternoon, the team can argue about it on Monday with something concrete in front of them.

ONE MORE DETAIL WORTH COPYING

The first run of the synthetic generator reported "first favourable window: 12 November". A cooling autumn sea passes down through the same temperature band that a warming spring sea passes up through, and the index scored them the same. The fix was one line: require a rising trend. The lesson is not the line. It is that a scientist read the output and said "that is wrong" within a minute of seeing it, which no amount of code review would have caught.

4. Define the target before touching data

The single most consequential document the team will write is one page long and contains no code: the target specification. Until it exists, every hour spent on data or models is speculative.

Questions it must answer

1 What is an event?

Candidates, from strictest to loosest: an NIFS closure announcement for a station (채취 금지); a PSP measurement above the regulatory limit of 0.8 mg/kg at a station; any PSP detection at a station; a red-tide bulletin naming *Alexandrium* in the area. Each gives a different number of positives per year, which drives everything downstream. Pick one primary and, if useful, one secondary.

2 At what spatial unit?

The NIFS monitoring station, not the bay. Bays are a presentation convenience. The eight hand-placed points on the map should become the actual station list with its history.

3 At what lead time?

"Will there be an event at station S within the next 7 days?" and "within 14 days" are different targets with different skill ceilings. Seven is operationally useful for sampling schedules. Fourteen is useful for industry. Start with one.

4 Probability or class?

Probability. Always. A class label hides the threshold decision inside the model; a probability leaves it with the people who bear the cost of being wrong.

5 What is the cost of each error?

A missed event may mean illness. A false alarm means lost harvest and lost trust in the tool. These costs are not symmetric and they are not the modeller's to set. Write them down, even roughly.

6 What would make you act?

If the forecast said 30% chance of an event at Jinhae in the next week, what would anyone do differently? If the answer is "nothing", the target is wrong or the users are.

The scientist writes this page. Claude Code can help draft it, by reading the NIFS bulletin format and summarising what is actually recorded, but the decisions in it are not delegable.

5. Data inventory: labels and features

Verified during the session: Open-Meteo answers from Seoul without a key, there is no public API for NIFS toxin bulletins, and KHOA buoy data and the NIFS red-tide bulletin are on data.go.kr behind a free key. Everything else in this table is from general knowledge and must be checked by the team before being relied on.

Labels: where the truth comes from

Source	What it gives	Access and caveats
NIFS 패류독소 발생현황 (shellfish toxin status)	Station, date, species, toxin type, concentration, closure status. The primary label.	Web pages, no API found. Request a historical extract from NIFS directly; a Korean team can. Expect format changes over years and station renames.
NIFS 적조속보 (red-tide bulletins)	Area, causative organism, cell density, water temperature, outlook. A secondary label and a feature.	data.go.kr API (dataset 15156746), free key. Text fields need parsing; Claude is good at that.
MFDS seafood recalls and inspection failures	Product-level enforcement that sometimes follows an event.	Already in the repo. A weak, lagged confirmation, not a label.

Features: what might predict it

Source	Variables	Access and caveats
KHOA 바다누리 coastal stations	Sea temperature, salinity, tide, in near real time; history by station.	data.go.kr key. Station coverage of the small bays is uneven; check which stations are actually near which monitoring sites.
KMA ASOS / AWS	Rainfall, wind, sunshine hours, air temperature, at dense land stations with long history.	data.go.kr key or the KMA API hub. Long, clean history: the best-behaved source here.
River discharge (WAMIS)	Freshwater input to the bays, a nutrient and stratification driver.	National water resources portal. Only some bays have a gauged river.
Satellite chlorophyll (GOCI-II via KIOST; Copernicus Marine)	Phytoplankton biomass as a leading indicator.	Cloud gaps, coastal pixel contamination. Worth it, but as the second feature set, not the first.
Open-Meteo historical archive	Reanalysis weather back decades; marine history shorter.	Keyless. A fallback for gaps and the fastest way to build a first feature table before the Korean keys arrive.

THE TRAP THAT RUINS MOST FIRST ATTEMPTS

Leakage. A closure announcement is dated when it is published, not when the water was sampled, and the sample lags the bloom. If the feature window reaches up to the announcement date, the model learns to see the bloom that caused the announcement, and its backtest looks wonderful. Anchor every feature window to the *sampling* date minus the lead time, and have the scientist confirm the lag structure before the first model is fitted.

6. The modelling ladder

Climb it one rung at a time. Each rung must beat the one below on the same backtest, or it does not ship. Skipping rungs is how teams end up with an impressive model that cannot beat "it is April".

1 Rung 0: climatology and persistence.

Probability of an event at station S in week W equals the historical frequency for that station and week. And: whatever happened last week happens this week. These cost nothing and are surprisingly hard to beat for a seasonal phenomenon. They are the denominator for every skill score that follows.

2 Rung 1: the heuristic index.

What the environmental tab does today, evaluated honestly for the first time. Does a high index precede events? With what lead? This is where the scientist's thresholds get their first contact with data, and where several of them will change.

3 Rung 2: logistic regression on lagged features.

Per station or pooled with station effects. Features: sea temperature and its 7- and 14-day changes, rainfall sums, wind, day-of-year as sine and cosine, last year's event history. Interpretable, fast, and usually a real improvement over rung 1. Produces a probability natively.

4 Rung 3: gradient-boosted trees.

LightGBM or similar, same features plus interactions it finds itself, with quantile objectives to produce `p10` and `p90` honestly. Often the practical ceiling for a dataset of this size. Requires more discipline about leakage and cross-validation, not more data science.

5 Rung 4: sequence models.

Only if rung 3 has clearly plateaued and the team has several seasons of dense data. For most monitoring programmes this rung is never reached and should not be.

At every rung, the output is written into the same contract from Section 3 and appears on the same page. The team can see rung 1 and rung 2 side by side on the cards if it wants to, by running two model versions and printing both names.

7. Evaluation is the scientist's job

A model is a claim about the future. Evaluation is how the claim is checked, and it is the part of the project that cannot be delegated to the people who built the model, nor to the AI that helped them.

How to backtest

- **Leave one season out.** Train on all other years, forecast the held-out spring day by day as if live, score it. Repeat for every year. Random cross-validation on daily rows is meaningless for this problem; it leaks tomorrow into today.
- **Forecast at the lead time, from the data available then.** No feature may use anything dated after the issue date. Build the harness so this is enforced by construction, not by care.
- **Per station and pooled.** A model can be excellent at Jinhae and useless at Yeosu. Report both.

What to measure

Metric	What it answers
Hit rate and false-alarm rate at the chosen threshold	Of real events, how many were flagged in time? Of flags, how many were wrong? The two numbers the harvesters and the regulators will ask about.
Brier score and skill against climatology	How good are the probabilities overall, and how much better than "it is April"?
Reliability diagram	When the model says 30%, does it happen about 30% of the time? If not, the probability is a decoration.
Lead-time curve	How does skill decay from 1 day ahead to 14? Tells you what the honest horizon is, which may be shorter than the contract's fourteen points.

THE TWENTY-CASE HABIT, SCALED UP

The primary guide's Exercise 5 asks the scientist to label twenty records by hand and compare them with the model's answers. The same habit applies here. Before the first backtest number is believed, the scientist should take one past season, walk through it week by week with the forecast in one hand and the NIFS record in the other, and write down every disagreement and what caused it. Most model improvements come from that walk, not from the metrics table.

8. Communicating a forecast

The page already draws the line the team must hold: observed conditions are one thing, forecasts are another, and neither is a toxin measurement or a safety claim. Keep every one of these when the real model arrives.

- **Bands, not lines.** The page draws `p10` to `p90` as a shaded band. A real model's band will be wider than the synthetic one, and that is a feature. A narrow band from a model with no skill is the most dangerous object in this project.
- **Provenance on the card.** Model name, issue date, data sources. Already printed; keep printing them.
- **The badge logic.** SYNTHETIC comes off only when the checklist in Section 11 is complete. Consider a second badge, EXPERIMENTAL, for the first real season.
- **Words the page may use.** Conditions, favourable, probability, monitor, outlook. Words it may not: safe, toxic, contaminated, clear. The advisory prompt in `src/kfs/advisory.py` already enforces this for Claude's text; the same list applies to humans writing labels.
- **Alignment with NIFS and MFDS.** The forecast supplements official monitoring; it never contradicts a published closure, and the page should link to the official bulletin wherever it shows a bay. A forecast tool that appears to second-guess the regulator will not survive its first disagreement.

When the real model ships, change the advisory

Today the Claude advisory deliberately does not see the forecast, because the forecast is invented. When a backtested model exists, add its output to the advisory input and add two sentences to the system prompt: what the model is, and how much weight to give it relative to observed conditions. Then re-run Exercise 5 on the advisories themselves.

9. Where Claude helps, and where it does not

Claude Code, the builder

Writes the data pipelines for each new source, with the same probe-first habit used for MFDS and Open-Meteo. Writes the feature code, the label-parsing code for NIFS bulletins, the backtest harness that enforces the no-future-data rule by construction, and the tests. Keeps `PLAN.md`, the docs and the prompt log current. Drafts the target specification from the bulletin format for the scientist to correct.

Prompt shape, unchanged: context, goal, constraints, "report back with a plan before changing code".

The Claude API, the worker

Writes the bilingual advisories. Parses free-text fields in red-tide bulletins into structured records. Explains a forecast in plain language to a harvester or an official. Summarises a season's disagreements between model and record for the review meeting.

Does **not** make the forecast. The statistical model is ordinary statistics: scikit-learn, LightGBM, a few hundred lines. Asking a language model to guess next week's toxin probability from numbers is slower, costlier, unreproducible and unbacktestable. Use the right tool.

Three prompts for the first week

1. *Read docs/PREDICTION.md and src/kfs/predict.py. Then read the NIFS shellfish toxin status page I am pasting below and tell me, as a table, exactly which fields it records per station and per date. Do not write code.*
2. *Draft a one-page target specification for predicting shellfish toxin events, using the six questions in Section 4 of the roadmap. Leave every decision as a blank for me to fill in, with the options and their consequences listed under each.*
3. *Build a backtest harness that takes a forecast function with the contract in PREDICTION.md and a labelled event table, runs leave-one-season-out, and reports hit rate, false-alarm rate, Brier skill against climatology and a reliability table. Make it impossible for a feature to see data after the issue date. Report back with a plan first.*

10. Team, roles and a twelve-week plan

Role	Owns	Typical person
Scientific lead	Target specification, label definitions, thresholds, the season walk-through, the decision to remove the SYNTHETIC badge.	The food-safety scientist.
Data engineer	Keys, pipelines, feature table, data quality, the repo.	The Gen MZ partner from the primary guide, working with Claude Code.
Modeller	Rungs 0 to 3, the backtest harness, calibration.	Someone with applied statistics; can be the engineer with Claude Code's help at rungs 0 to 2.
Domain reviewers	Reading forecasts against lived experience each week of the season.	NIFS or local fisheries contacts, harvesters.

Twelve weeks

Weeks	Deliverable
1–2	Target specification signed by the scientific lead. data.go.kr keys. Historical NIFS extract requested. Station list replaces the eight hand-placed bays.
3–5	Label table and feature table, with the lag structure written down. Rung 0 baselines and the backtest harness. First honest number: how hard is this?
6–8	Rung 1 evaluated; thresholds revised. Rung 2 fitted and backtested. Season walk-through for one past year.
9–10	Rung 3 with quantile outputs. Reliability checked. Lead-time curve sets the honest horizon.
11–12	Best rung wired into the contract. Advisory prompt updated. Checklist in Section 11 worked through. Decision on the badge. Everything in the repo.

This is a plan for a first honest model, not a finished product. A second season of live forecasts, reviewed weekly, is what turns it into a tool.

11. Checklist before switching off the SYNTHETIC badge

1. A written target specification exists and the scientific lead has signed it.
2. Labels come from NIFS records, with the sampling-to-publication lag documented.
3. Every feature is anchored to the issue date minus the lead time, enforced by the harness.
4. Leave-one-season-out backtest over at least three seasons, per station and pooled.
5. The model beats climatology and persistence on Brier skill, and the scientific lead has seen the reliability diagram.
6. Hit rate and false-alarm rate at the operating threshold are written down next to the cost of each error.
7. The season walk-through has been done for at least one held-out year and its disagreements are recorded.
8. `model` names a version that is tagged in the repository.
9. The advisory prompt has been updated and its outputs reviewed for the forbidden words.
10. The page links to the official NIFS bulletin from every bay.
11. Someone outside the team has read the page and could not mistake a forecast for a measurement.

AND THEN

Switch `synthetic` to `false`, consider an EXPERIMENTAL badge for the first live season, and keep the weekly walk-through going. The model is not the product. The habit of checking it is.

12. Glossary

Target	The thing being predicted, defined precisely: event type, place, lead time, probability.
Label	The recorded truth for past dates, used to train and test. Here, NIFS toxin records.
Feature	An input the model may use: sea temperature, rainfall, day of year.
Lead time	How far ahead the forecast looks. Skill falls as it grows.
Leakage	Letting the model see information from after the forecast date. Makes backtests lie.
Backtest	Forecasting the past as if live, then scoring against what happened.
Climatology	The long-run average for that place and time of year. The baseline every model must beat.
Persistence	Forecasting that tomorrow equals today. The other baseline.
Quantile (p10, p50, p90)	Values the outcome falls below 10%, 50% and 90% of the time. Together, an uncertainty band.
Brier score	Mean squared error of a probability forecast. Lower is better. Compare to climatology for skill.
Reliability	Whether "30%" events happen about 30% of the time.
Hit rate / false-alarm rate	Share of real events flagged; share of flags that were wrong.
Logistic regression	A simple, interpretable model that outputs a probability from weighted features.
Gradient boosting	An ensemble of small decision trees; often the practical ceiling on tabular data.
Contract	The fixed output shape every model version must produce. Section 3.
Synthetic	Generated to look plausible, with no claim to skill. Badged as such until proven otherwise.

Sources and attributions. Data on the page: 식품의약품안전처 식품안전나라; Open-Meteo (CC BY 4.0); KOSTAT boundaries via southkorea-maps; Esri World Street Map tiles; Leaflet. Model used for advisories: Claude Opus 5.5. This document was drafted by Claude Code from the session that built phases 2 and 3 and reviewed by its author. Data-source availability other than Open-Meteo, data.go.kr listings and the absence of a public NIFS toxin API was not verified in-session and must be checked by the team.